

Pañuelo al Viento

Documentación de sistema

05 · Referencia técnica

Versión del producto **0.1.0+1**
Commit **6efae74** · 2026-08-25
Rama **main**

Generado desde los Markdown de docs/system-documentation/ con tool/build_docs_pdf.py.
La versión y el commit se leen del repositorio; no están escritos a mano.

Contenido de este documento

lib/domain/curriculum.dart

lib/data/curriculum_repository.dart

lib/data/progress_repository.dart

lib/state/app_state.dart

lib/screens/rhythm_lab_screen.dart

lib/screens/lesson_screen.dart

lib/widgets/movement_diagram.dart

Herramientas · tool/app_version.mjs

Continuar por

05 · Referencia técnica

Catálogo de consulta. Para cada símbolo relevante: firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién lo llama, a quién llama y **riesgo al modificarlo**.

Los identificadores se citan en su forma original.

lib/domain/curriculum.dart

enum ActivityType

```
enum ActivityType { discover, observe, move, listen, create, reflect }
```

Naturaleza pedagógica de una actividad. **La interfaz no ramifica por este valor**: hoy solo documenta la intención con la que el currículo equilibra la secuencia. Reparto real en las 72 actividades: `move` 22, `reflect` 18, `observe` 16, `create` 7, `discover` 5, `listen` 4.

Riesgo al modificar: bajo hoy. Añadir un valor no rompe nada porque no hay switch exhaustivo sobre él. Pasaría a ser alto en cuanto alguien lo use para decidir un icono o un color.

ActivityType activityTypeFromJson(String value)

Aspecto	Detalle
Parámetro	<code>value</code> — el campo <code>type</code> del JSON.
Retorno	El valor coincidente, o <code>ActivityType.discover</code> si no hay ninguno.
Excepciones	Ninguna.

Degrada en silencio. Es la única tolerancia del dominio, y es deliberada: un `type` mal escrito no debe impedir que una clase se abra. El control estricto vive en los validadores.

Riesgo: hacerlo lanzar convertiría una errata de contenido en un fallo de arranque de toda la aplicación.

class LearningActivity

Campos final: `type`, `title`, `instruction`, `minutes`. Constructor `const`.

factory `LearningActivity.fromJson(Map<String, dynamic> json)` — lanza `TypeError` si falta un campo o el tipo no coincide. Los validadores comprueban antes que los minutos de las tres actividades sumen el `durationMinutes` de su clase.

class Lesson

Campos final: `id`, `order`, `title`, `durationMinutes`, `objective`, `why`, `diagram`, `activities`, `challenge`, `safety`, `accessibility`, `teacherTip`.

`order` es la posición **global** entre las 24, no dentro del nivel: es el número de la tarjeta y el que decide cuál es la próxima clase.

`diagram` es una clave que interpreta `MovementDiagram`. Valores presentes en 0.1.0 y su frecuencia: `circle` 4, `eight` 4, `pair` 4, `steps` 4, `free` 3, `semicircle` 3, `wave` 2.

`activities` se construye con `.toList(growable: false)`: no se puede añadir ni quitar después.

Riesgo al añadir un campo: alto en cadena. Hay que tocar `fromJson`, el JSON de las 24 clases, los dos validadores y probablemente `curriculum_model_test.dart`. Un campo nuevo obligatorio hace fallar el arranque hasta que las 24 lo tengan.

class LearningLevel

Campos: `id`, `order`, `title`, `subtitle`, `emoji`, `lessons`. Los validadores exigen exactamente 3 clases por nivel.

class Curriculum

Campos: `version`, `title`, `audience`, `levels`.

List<Lesson> get lessons

```
List<Lesson> get lessons =>
  levels.expand((level) => level.lessons).toList(growable: false)
  ..sort((a, b) => a.order.compareTo(b.order));
```

Aplana y ordena por `order`. **Construye y ordena una lista nueva en cada llamada.** Con 24 elementos es irrelevante, pero conviene saberlo: lo invocan `AppState.totalCount`, `AppState.nextLesson` y `AppState.completeLesson`, y este último recorre además el resultado para construir un `Set`.

Riesgo: usarlo dentro de un `build` que se repinte a 60 fps. Hoy no ocurre.

lib/data/curriculum_repository.dart

Future<Curriculum> load()

Aspecto	Detalle
---------	---------

Retorno	El currículo completo.
Excepciones	<code>FlutterError</code> si el activo no está en el bundle; <code>FormatException</code> si el JSON es inválido; <code>TypeError</code> si falta un campo. Ninguna se captura aquí.
Efectos	Lectura del bundle de activos. Ninguna escritura.
Llamado por	<code>AppState.load()</code> y tres archivos de prueba.
Llama a	<code>rootBundle.loadString</code> , <code>jsonDecode</code> , <code>Curriculum.fromJson</code> .

Usa `rootBundle` y no el sistema de archivos deliberadamente: el contenido viaja dentro de la instalación y se lee igual en Android, en Windows y en las pruebas. Esa elección es también lo que mantiene al producto sin acceso general a archivos del dispositivo.

Riesgo: capturar la excepción aquí destruiría la pantalla de recuperación de `main`, que es lo que evita que una niña o un niño vea una pantalla en blanco.

`lib/data/progress_repository.dart`

abstract interface class ProgressStore

```
Set<String> readCompletedLessonIds();
Future<void> writeCompletedLessonIds(Set<String> lessonIds);
Future<void> clear();
```

Contrato del almacenamiento. **Regla:** los dos métodos asíncronos *lanzan* cuando la operación no se confirma. Devolver sin más sería indistinguible del éxito.

Existe únicamente para poder inyectar un almacén que falle a voluntad. Sin esta interfaz, las tres pruebas de `test/app_state_test.dart` serían imposibles de escribir, y con ellas se perdería la única red de seguridad de la regla transaccional.

`ProgressRepository(SharedPreferences preferences)`

Clave: `static const _completedKey = 'completed_lessons_v1'`. El sufijo `_v1` reserva la posibilidad de migrar el formato. Cambiarla deja huérfano el avance guardado en los dispositivos ya instalados.

`Set<String> readCompletedLessonIds()`

Síncrono. Devuelve conjunto vacío si la clave no existe. No lanza.

`Future<void> writeCompletedLessonIds(Set<String> lessonIds)`

Ordena, escribe y lanza `StateError('No fue posible guardar el avance en este dispositivo.')` si `setStringList` devuelve `false`.

El ordenamiento hace comparables dos ejecuciones con el mismo avance, lo que `test/progress_repository_test.dart` aprovecha.

Riesgo: eliminar el `throw` rompe silenciosamente toda la garantía transaccional. `AppState` no tiene forma de saber que la escritura falló.

`Future<void> clear()`

Lanza `StateError`('No fue posible borrar el avance de este dispositivo.') si `remove` devuelve `false`.

REQUIERE VALIDACIÓN: `SharedPreferences.remove` sobre una clave inexistente devuelve `true` en las implementaciones habituales, pero el comportamiento no está documentado en el repositorio ni se probó en un dispositivo real. Si devolviera `false`, reiniciar un avance ya vacío lanzaría.

`lib/state/app_state.dart`

`AppState({required CurriculumRepository curriculumRepository, required ProgressStore progressRepository})`

Extiende `ChangeNotifier`. Nótese la asimetría de los tipos: el repositorio de currículo se pide **concreto** y el de progreso se pide por **interfaz**. Solo el segundo necesita ser sustituible, porque solo el segundo puede fallar de una forma que haya que probar.

`Curriculum get curriculum`

Lanza `StateError`('El currículo todavía no está cargado.') si se consulta antes de `load()`. Intencionado: un `null` silencioso se propagaría a toda la interfaz.

`Set<String> get completedLessonIds`

Devuelve `Set.unmodifiable(...)`. Nadie fuera de la clase puede alterar el avance sin pasar por los métodos que persisten. Propiedad pequeña y valiosa.

`int get completedCount` · `int get totalCount` · `double get progress`

`progress` protege la división: `totalCount == 0 ? 0 : completedCount / totalCount`. `totalCount` invoca `curriculum.lessons`, que ordena.

`Lesson? get nextLesson`

Primera clase no completada en orden global, o `null`. Es un avance **por hueco**, no por posición: completar la 5 antes que la 3 deja la 3 como próxima. La ruta no bloquea clases, así que este valor es una sugerencia, no una puerta.

Llamado por `HomeTab.build`.

`Future<void> load()`

Aspecto	Detalle
Efectos	Asigna <code>_curriculum</code> , filtra <code>_completedLessonIds</code> , <code>notifyListeners()</code> .
Excepciones	Propaga las de <code>CurriculumRepository.load()</code> .
Llamado por	<code>main()</code> y cinco pruebas.

La línea que hay que entender:

```
_completedLessonIds =  
  _progressRepository.readCompletedLessonIds().intersection(validIds);
```

Cruza lo guardado con los identificadores que el currículo actual declara válidos. Una clase retirada o renombrada en una versión posterior deja de contar como completada y no infla el porcentaje. El descarte es **silencioso y no**

reescribe el almacén: los identificadores desconocidos siguen en disco hasta la próxima escritura, momento en el que desaparecen. `NO DOCUMENTADO EN EL REPOSITORIO` fuera de una frase en `../ARCHITECTURE.md`.

`bool isCompleted(String lessonId)` · `double levelProgress(LearningLevel level)`

Consultas puras. `levelProgress` devuelve 0 para un nivel sin clases, caso que el currículo actual no produce porque los validadores exigen 3.

`Future<void> completeLesson(String lessonId)`

Aspecto	Detalle
Excepciones	<code>ArgumentError</code> si el identificador no existe en el currículo. <code>StateError</code> propagado si la escritura no se confirma.
Efectos	Escribe en <code>SharedPreferences</code> , actualiza el conjunto, notifica. En ese orden.
Llamado por	<code>LessonScreen._complete()</code> y dos pruebas.

Si la clase ya estaba completada, retorna sin escribir: repetir es parte del diseño pedagógico y no debe generar tráfico ni notificaciones.

Riesgo al modificar: **el más alto del repositorio.** Publicar `_completedLessonIds` antes del `await` deja la interfaz mintiendo cuando el disco falla. Las tres pruebas de `test/app_state_test.dart` existen exactamente para impedir ese cambio.

`Future<void> resetProgress()`

Mismo orden: borrar, luego vaciar en memoria, luego notificar. Si el borrado falla, el avance visible se mantiene. Es preferible a mostrar cero clases y que reaparezcan al reiniciar. Llamado por `ProgressTab._confirmReset` tras confirmación.

`lib/screens/rhythm_lab_screen.dart`

Estado interno de `_RhythmLabScreenState`

Campo	Tipo	Inicial	Papel
<code>_timer</code>	<code>Timer?</code>	<code>null</code>	Temporizador de un disparo del próximo pulso.
<code>_clock</code>	<code>Stopwatch</code>	<code>detenido</code>	Referencia monotónica.
<code>_scheduledTick</code>	<code>int</code>	<code>0</code>	Cuántos pulsos se han programado desde el último reinicio.
<code>_bpm</code>	<code>double</code>	<code>84</code>	Velocidad. El deslizador va de 60 a 120 con 12 divisiones, es decir pasos de 5; 84 no cae en ninguna división.
<code>_pulse</code>	<code>int</code>	<code>0</code>	Pulso actual, 0–5.

<code>_playing</code>	<code>bool</code>	<code>false</code>	Si el metrónomo corre.
<code>_sound</code> · <code>_vibration</code>	<code>bool</code>	<code>true</code>	Interruptores de salida. Encendidos por defecto, pero inertes hasta pulsar Comenzar .
<code>_pattern</code>	<code>_PulsePattern</code>	<code>sixEight</code>	Agrupación.

`List<int> get _accents`

`sixEight` → [0, 3] (3+3), `threeFour` → [0, 2, 4] (2+2+2). Un `switch` de expresión exhaustivo sobre el enum: añadir un valor a `_PulsePattern` **rompe la compilación aquí**, que es el comportamiento deseado.

`void _toggle()`

Si suena: cancela, para y reinicia el reloj, `_playing` = `false`, `_pulse` = 0. Si no: `_playing` = `true`, emite el pulso 0 inmediatamente y arranca la planificación. El primer pulso suena antes de esperar un intervalo, que es lo que hace que Comenzar se sienta inmediato.

`void _startSchedule()`

Cancela el temporizador vigente, pone `_scheduledTick` a 0, reinicia el `Stopwatch` y programa. Se llama también al mover el deslizador mientras suena, para que el cambio de tempo se note de inmediato.

`void _scheduleNextPulse()`

El corazón del laboratorio:

```
final microsPerPulse = 60000000 / _bpm;
final targetMicros = (microsPerPulse * (_scheduledTick + 1)).round();
final remainingMicros = targetMicros - _clock.elapsedMicroseconds;
_timer = Timer(Duration(microseconds: remainingMicros > 0 ? remainingMicros : 0), ...);
```

Cada pulso se agenda contra un **instante absoluto** de la línea temporal, no contra el momento en que se ejecutó el anterior. Un retraso acorta el intervalo siguiente —o lo dispara de inmediato con espera cero— y la serie vuelve a su sitio.

Guardas: retorna si `!_playing` antes de programar; dentro de la devolución de llamada comprueba `!mounted` || `!_playing` antes de continuar.

Riesgo: sustituirlo por `Timer.periodic` reintroduce una deriva que solo se nota tras varios minutos, que es exactamente cuando importa.

`void _emitPulse()`

Clic en cada pulso si `_sound`; háptica **solo en accents** si `_vibration`, para que la vibración marque la agrupación y no sea un zumbido continuo. Ambas pueden ser ignoradas por el equipo y esa ausencia no se trata como error.

`void dispose()`

Cancela el temporizador y para el reloj. **Solo se ejecuta al destruirse el widget**, y el `IndexedStack` de `HomeShell` no lo destruye al cambiar de pestaña. Consecuencia verificada en pruebas y registrada en [15 · Riesgos](#).

`LessonScreenState._activityChecks`

`List<bool>` de longitud fija igual al número de actividades. **Estado local, no persistente**. Salir de la clase sin completarla lo descarta. Es coherente con el inventario de `../PRIVACY.md`: no se guarda el detalle de la práctica.

`Future<void> _complete()`

`await completeLesson` primero, comprobación de `mounted`, diálogo de celebración, `setState` final. El orden importa: si la escritura falla, la excepción sube y no se llega a felicitar por algo que no quedó guardado.

`void _finishRepeat()`

No toca la persistencia. Limpia las casillas y muestra un `SnackBar` avisando de que el avance anterior sigue guardado, porque desmarcar tres casillas parece un retroceso si nadie lo dice.

`lib/widgets/movement_diagram.dart`

`MovementDiagram({required String pattern})`

Envuelve un `CustomPaint` de proporción 16:9 en un `Semantics` con `image: true` y una etiqueta descriptiva.

`String _description(String value)`

`switch` de expresión con ramas para `circle`, `eight`, `semicircle`, `steps`, `pair`, `wave` y un comodín. **No tiene rama para `free`**, que cae en el comodín: «Espacio personal para practicar un movimiento libre», descripción que resulta correcta por casualidad.

`MovementPainter.paint(Canvas, Size)`

Ramas de dibujo: `circle`, `eight` (paramétrica, 210 segmentos), `semicircle`, `steps`, `wave`, y `default` (una línea entre las dos figuras). `pair` y `free` comparten esa rama por defecto.

Todas las medidas son fracciones de `size`. Los colores oscuros se eligen a mano en lugar de tomarse del `ColorScheme` porque el lienzo tiene fondo propio y debe contrastar con él, no con la tarjeta que lo contiene.

Detalle no obvio: la rama `steps` muta el `Paint` compartido a `PaintingStyle.fill` y lo restaura a `stroke` después; la rama `default` muta `strokeWidth` a 2 y **no lo restaura**. Es inocuo porque el `Paint` es local a cada llamada de `paint`, pero es el tipo de detalle que se vuelve un error al extraer el objeto a un campo.

`bool shouldRepaint(covariant MovementPainter oldDelegate)`

Repinta si cambió `pattern` o `darkMode`. Correcto y ajustado.

Herramientas · `tool/app_version.mjs`

`export function readAppVersion(pubspecPath = 'pubspec.yaml')`

Aspecto	Detalle
Retorno	<code>{ version, build, full }</code> . Para <code>version: 0.1.0+1</code> → <code>{ '0.1.0', '1', '0.1.0+1' }</code> .
Excepciones	Error si no encuentra una línea <code>version: X.Y.Z+N</code> .

Lo usan

`validate_repository.mjs`, `verify_apk.mjs`, `build_site.mjs` y `release.yml`.

También funciona como CLI: `node tool/app_version.mjs version|build|full`. La detección de invocación directa compara `import.meta.url` con `pathToFileURL(process.argv[1])`.

Es el símbolo con más consecuencias del repositorio: todo el sistema de versiones depende de que esta expresión regular siga coincidiendo.

Continuar por

- [06 · Explicación profunda](#) para ver estos símbolos en movimiento.
- [19 · Matriz de trazabilidad](#) para ir de una funcionalidad a su prueba en una sola fila.